

Installing Kali and Writing a Script

- **Installing Kali Linux**

- Kali Install Selection Process:
 - For my Kali installation I decided to use virtual box and a pre-built virtual image. I did this for a few reasons, one virtual box is running only on my local machine and doesn't really require an internet connection to use if I have all my files already. This means I can work offline and anywhere. I chose the pre-built image because virtual box is one of the supported images so I felt confident it would be easy to install.
- Kali Install using Virtual Process
 - The installation process was very simple. I downloaded virtual box from oracle and downloaded the virtual box kali image from the kali website. Installing the kali image is as simple as starting virtual box and clicking "add" then navigating to the virtual box .vbox file that I downloaded. Once the virtual box finishes booting one logs in using "kali:kali"
- Displaying Username
 - This part is slightly out of order as you can see in the video below the whoami returns "James" which I detail creating next. Running whoami is just opening a prompt and typing the word "whoami" and then enter. It returns the username "James"
- Creation of new Sudo Capable User "James"
 - I'm not sure if this step is part of the assignment but it seemed important to me that I could create other users than the default, and obviously I want that account to have sudo access. This is where I ran into my first problem of the assignment, I googled "linux adding sudo user" This took me to a page online that listed the commands for adding a new user and then adding them to the sudo capable group. After creating my user James, I tried to logout of Kali and back into James. However it failed, I could see that when the password was wrong, I got an error message. But when it was right the system appeared to attempt to login and would take me back to the login screen. I finally noticed that several things were different about the James account vs the kali account. Most notable, the new James folder was missing all of its "home" directories. A quick google search of that and you find pages talking about how for the kali distro the command to add a user is "useradd -m username" not "useradd

username” like most linux distros. After repeating the steps with the flag, I was then able to login with my new account.

- **Kali Linux Script**

- Kali Script Selection Process
 - The scripting language that I selected is bash. I did so because the assignment seems to lend itself towards a command line scripting language like bash. Also I choose bash because it seems like it will be useful down the road in this course and I would like the practice.
- Kali Script Implementation
 - The initial script only had to print all the usernames in this kali instance, and the prompt specifically mentions the shadow file with a link (Linux-user-administration, 2025). Scrolling down the page you eventually find “awk -F:' '{ print \$1}' /etc/passwd”. I used vi to create a script called printUsers.sh and copied the command in. Before running the script, I had to change the permission of the file with “chmod +x printUsers.sh” I then ran the script (Linux-user-administration, 2025) and got back the list of usernames.
- Kali Bonus Script Implementation
 - For the bonus problem I was a little unsure how to sort login accounts, so I asked ChatGPT for a guide. (chatGPT, 2025). Here is a screenshot of my prompt and response

Please provide a guide to a bash script to categorizes usernames into "login" and "non-login" accounts.



Here's a guide to creating a bash script that categorizes usernames into "login" and "non-login" accounts based on the shell they use in `/etc/passwd`. Typically, login accounts have valid shells like `/bin/bash` or `/bin/zsh`, while non-login accounts use shells like `/sbin/nologin` or `/bin/false`.

Steps to Create the Script

1. **Script Purpose:** Categorize system users into "login" and "non-login" accounts.
2. **Source of Data:** The `/etc/passwd` file contains user account information.
3. **Logic:**
 - Parse each line of `/etc/passwd`.
 - Extract the username (first field) and shell (last field).
 - Categorize based on the shell.

Upon inspection of the shadow file I also noted that all the none log in accounts have a "!" or "*" as their password whereas actual login accounts all had a hash. I eventually figured it would be good to check against the password as well.

The pseudo code for my script went as follows.

I use paste to combine together the passwd and shadow file to get all the data columns that I will need. I then loop over each entry using while and read in the username, shell, and password. Using an if statement I first check if the shell type matches any of bin/bash, bin/zsh, bin/sh as this is the first step for confirming this is a login type. I would later find a mistake in this part, but I will return to that. If the shell type isn't one of those then I else to printing out that this user is a non-login account.

This attempt worked, mostly, however I noticed that root was being flagged as a login account because of its shell type. However given that you can't use it to login into kali itself I figured it should not be. I now needed to add a nested if for the first if that also checked the password type so that root would be set as none login instead.

Overall, my scripting experience was "meh", the actual logic of what I was trying to implement was very straightforward. I have had some experience with bash before, but I had

forgotten how unbelievably picky it can be about syntax and formatting, I may consider python in the future for ease of use. But this was probably good experience with Bash